

Sistemas Operativos

Ayudantía 8: File Systems

Profesor: Yerko Ortiz

Ayudante: Diego Banda



Contacto



diego.banda@mail.udp.cl



darkclouds



github.com/DiegoBan/SO-2025-2

**Ayudante
Cátedra**

**Ayudante
Corrector**



enzo.rodriguez@mail.udp.cl



hooqqi



+56 9 4906 1114

File System

¿Que es un File System?

Un sistema de archivos es el componente del sistema operativo encargado de gestionar cómo se almacenan, organizan y acceden los datos en un dispositivo de almacenamiento (como discos duros, SSD, USB, etc.). Está compuesto por dos partes principales:

1. Colección de archivos: cada archivo contiene datos relacionados, como documentos, imágenes, programas, entre otros.
2. Estructura de directorios: organiza los archivos en una jerarquía (carpetas y subcarpetas), facilitando su búsqueda, acceso y administración.

Además, el sistema de archivos se encarga de llevar el control de atributos como el nombre del archivo, permisos de acceso, tamaño y fechas de creación/modificación. Algunos ejemplos de sistemas de archivos comunes son FAT32, NTFS, ext4 y APFS.

Links

- El soft link se identifica con una "l" (letra ele) al principio en el listado de `ls -l`, mientras que el hard link aparece con un "-", como cualquier archivo normal.
- Tanto el hard link como el archivo original tienen un contador de enlaces (link count). Si ves 2, significa que existen dos referencias (nombres) al mismo contenido. Si se creara un tercer hard link, el contador subiría a 3.
- El soft link indica explícitamente hacia dónde apunta (aparece con `→ destino`).
- Si se elimina el archivo original (`Solemne_1.ppt`), el soft link (`link_Solemne.ppt`) se convierte en un dead link (enlace roto).
- Un soft link es simplemente un archivo especial que contiene una ruta, mientras que un hard link comparte directamente el contenido con el archivo original.
- Aunque en la terminal ambos se vean como archivos normales, el hard link no es una "copia", sino otra entrada al mismo i-nodo.

Implementacion de un File system

- **Master Boot Record (MBR):**

Es el primer sector del disco (conocido como Sector 0), y se carga al iniciar el PC. Contiene el Boot Loader, que se encarga de iniciar el sistema operativo.

- **Partition Table:**

Está ubicada dentro del MBR y contiene información sobre las particiones del disco: su ubicación, tamaño, tipo de sistema de archivos, etc.

- **Boot Control Block:**

Presente en cada partición, permite cargar el sistema operativo desde esa partición específica. Suele ser parte del primer bloque de la partición.

- **Volume Control Block:**

Almacena información sobre el formato del sistema de archivos de la partición: tipo de FS (ext4, NTFS, etc.), número total de bloques, tamaño de bloque, entre otros datos clave.

- **File Control Block (FCB):**

Contiene los metadatos de cada archivo, como el nombre, tamaño, permisos, ubicación en disco, fechas de acceso/modificación, etc.

- **Root Directory:**

Es el directorio raíz de la partición, donde comienza la estructura jerárquica del sistema de archivos.

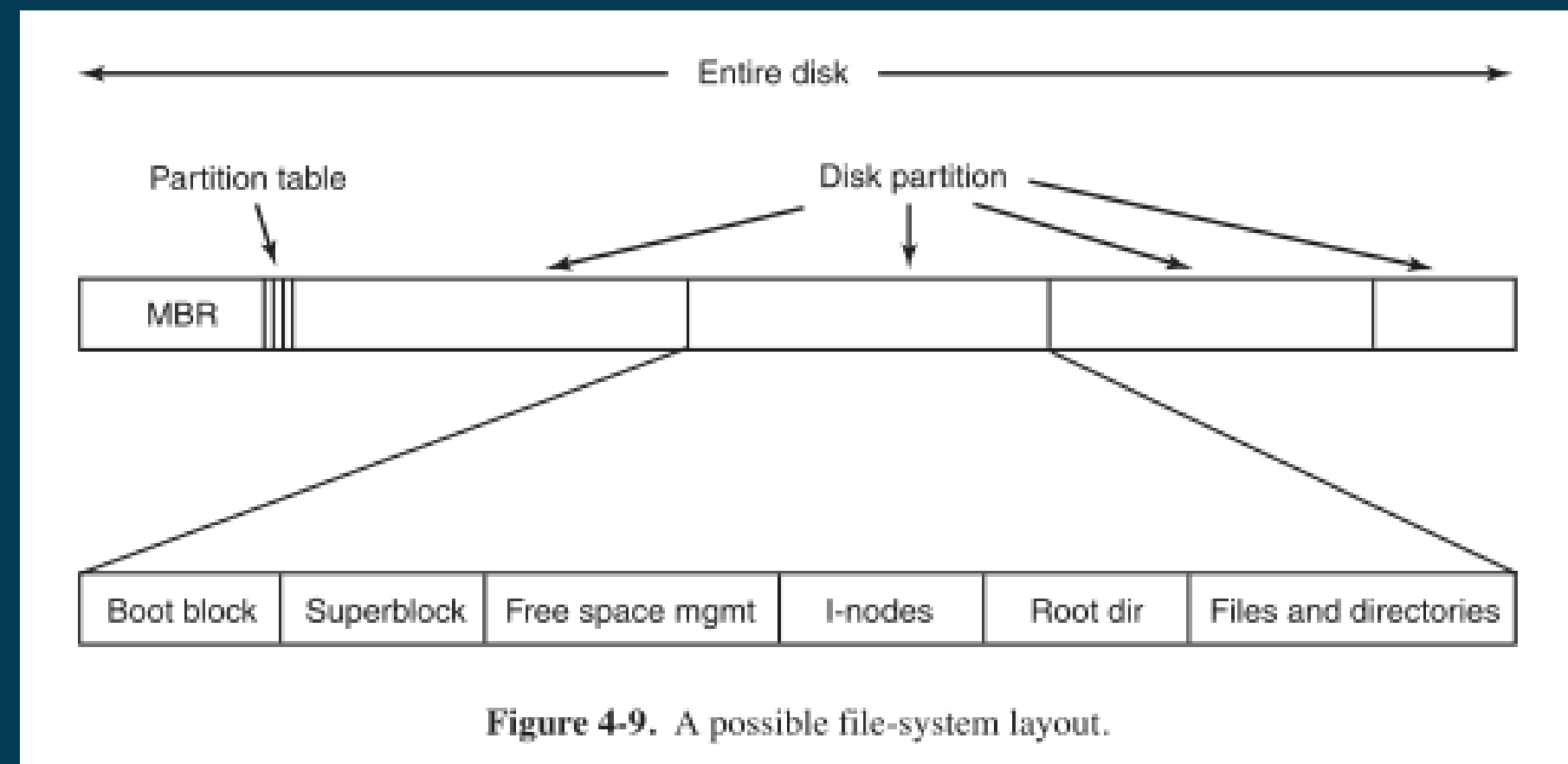
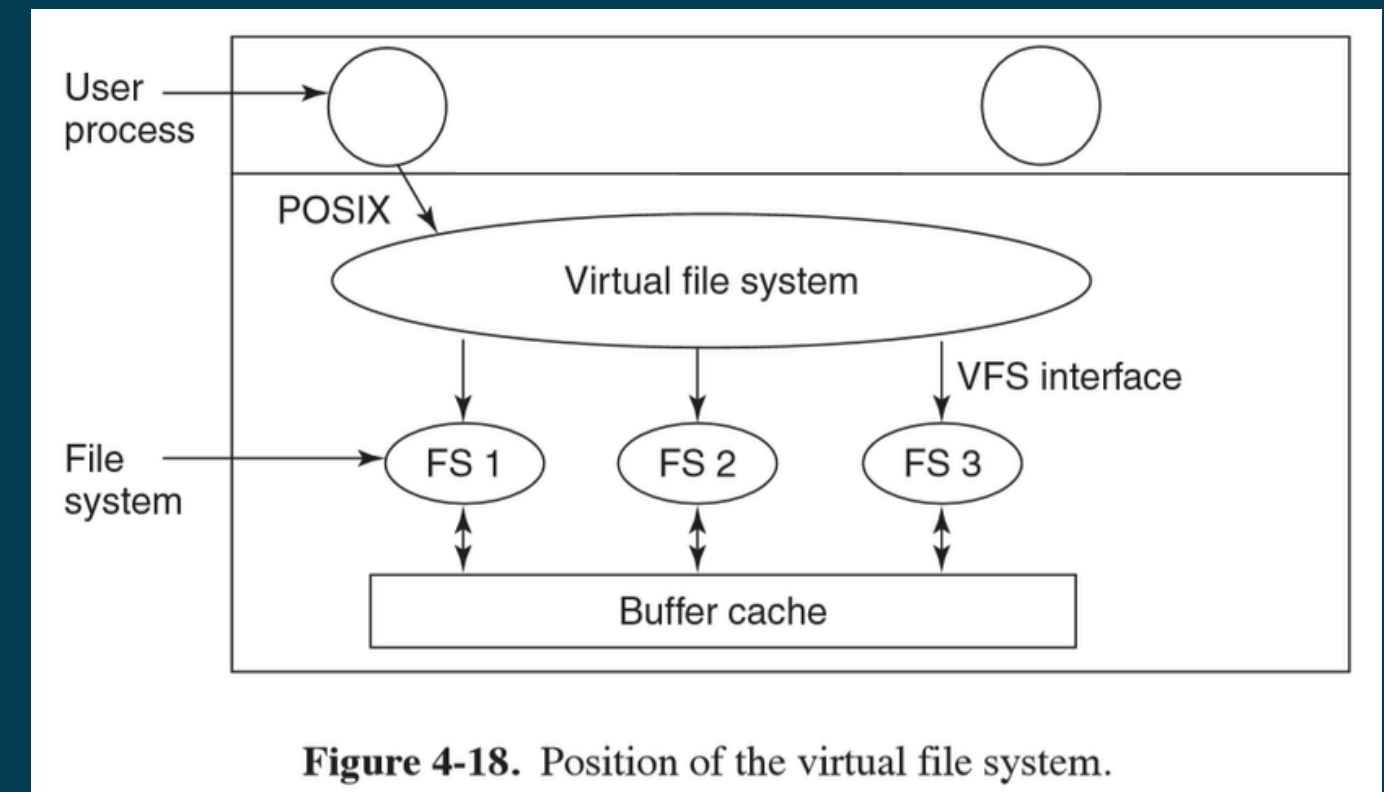


Figure 4-9. A possible file-system layout.

VFS (Virtual File System)

- Es una capa de abstracción que permite al sistema operativo interactuar con distintos tipos de sistemas de archivos de forma uniforme.
- Gracias a VFS, cualquier sistema de archivos que lo implemente (como ext4, FAT32, NTFS, etc.) puede integrarse al sistema operativo sin necesidad de cambiar la forma en que los programas acceden a los archivos.
- Proporciona una interfaz común y unificada, facilitando la interoperabilidad entre sistemas de archivos heterogéneos.
- VFS utiliza estructuras llamadas v-nodes (virtual nodes), que representan archivos o directorios de forma abstracta.
- Cada v-node apunta a funciones específicas del sistema de archivos real, que implementan operaciones como open, read, write, etc.
- Importante: Los v-nodes no son equivalentes a los i-nodos. Mientras que los i-nodos contienen información física de archivos en sistemas como ext4, los v-nodes son estructuras virtuales gestionadas por VFS para abstraer esa información.



Lectura de un Archivo con VFS

1. Inicio desde el proceso:

Una aplicación solicita la lectura de un archivo mediante una syscall (por ejemplo, read()).

2. Uso del File Descriptor:

El sistema busca en la tabla de procesos el file descriptor, que identifica el archivo abierto por ese proceso.

3. Acceso al v-node:

El file descriptor apunta a un v-node en el VFS, que representa el archivo de forma abstracta.

Este v-node contiene:

- Información del archivo.
- Punteros a funciones del sistema de archivos real (read, write, etc.).

4. Invocación de syscall específica:

El VFS llama a la función correspondiente del sistema de archivos real (por ejemplo, read()).

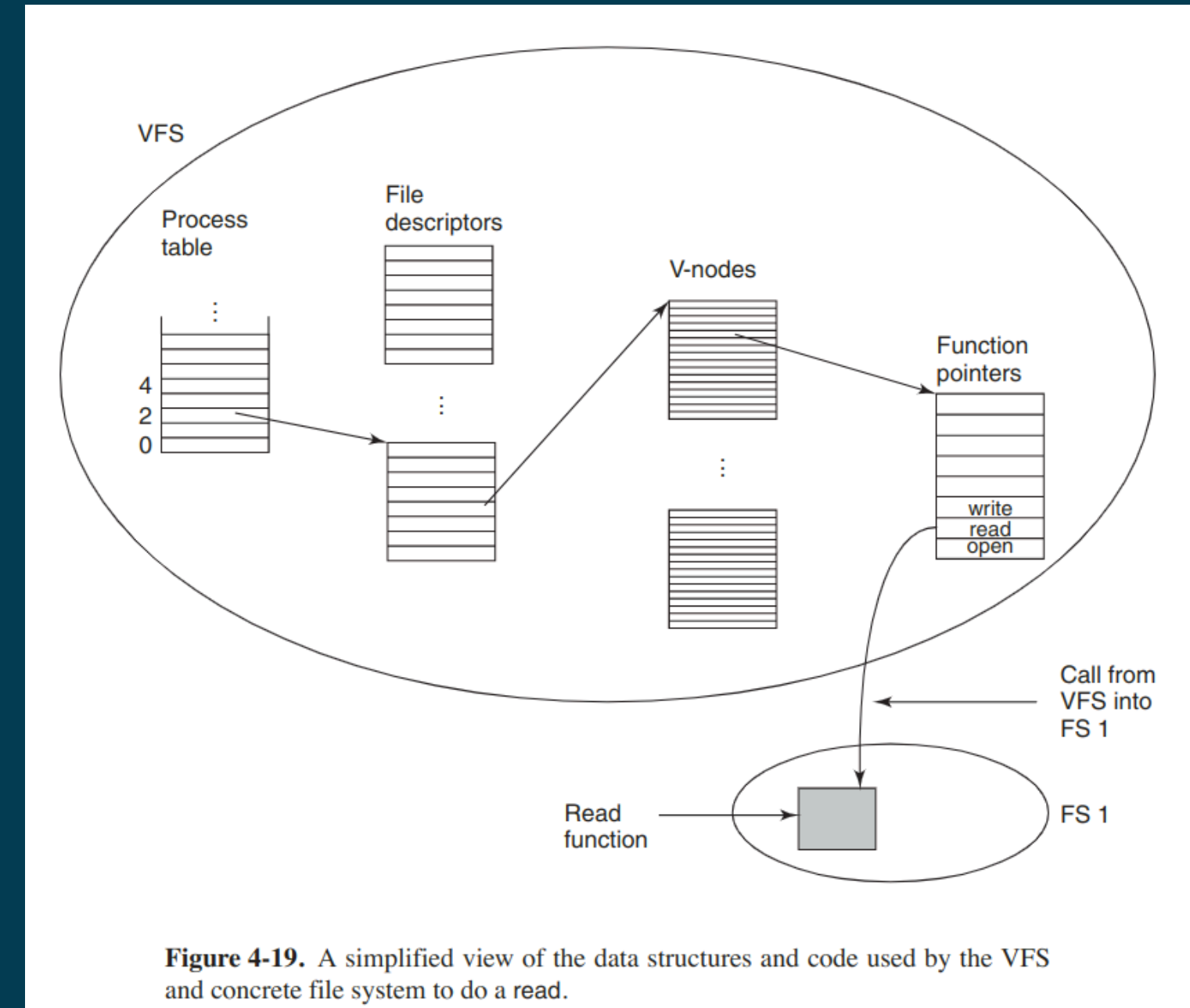
5. Lectura desde el FS real:

La función read() del sistema de archivos accede al dispositivo de almacenamiento y retorna los datos al proceso.

Ejemplo:

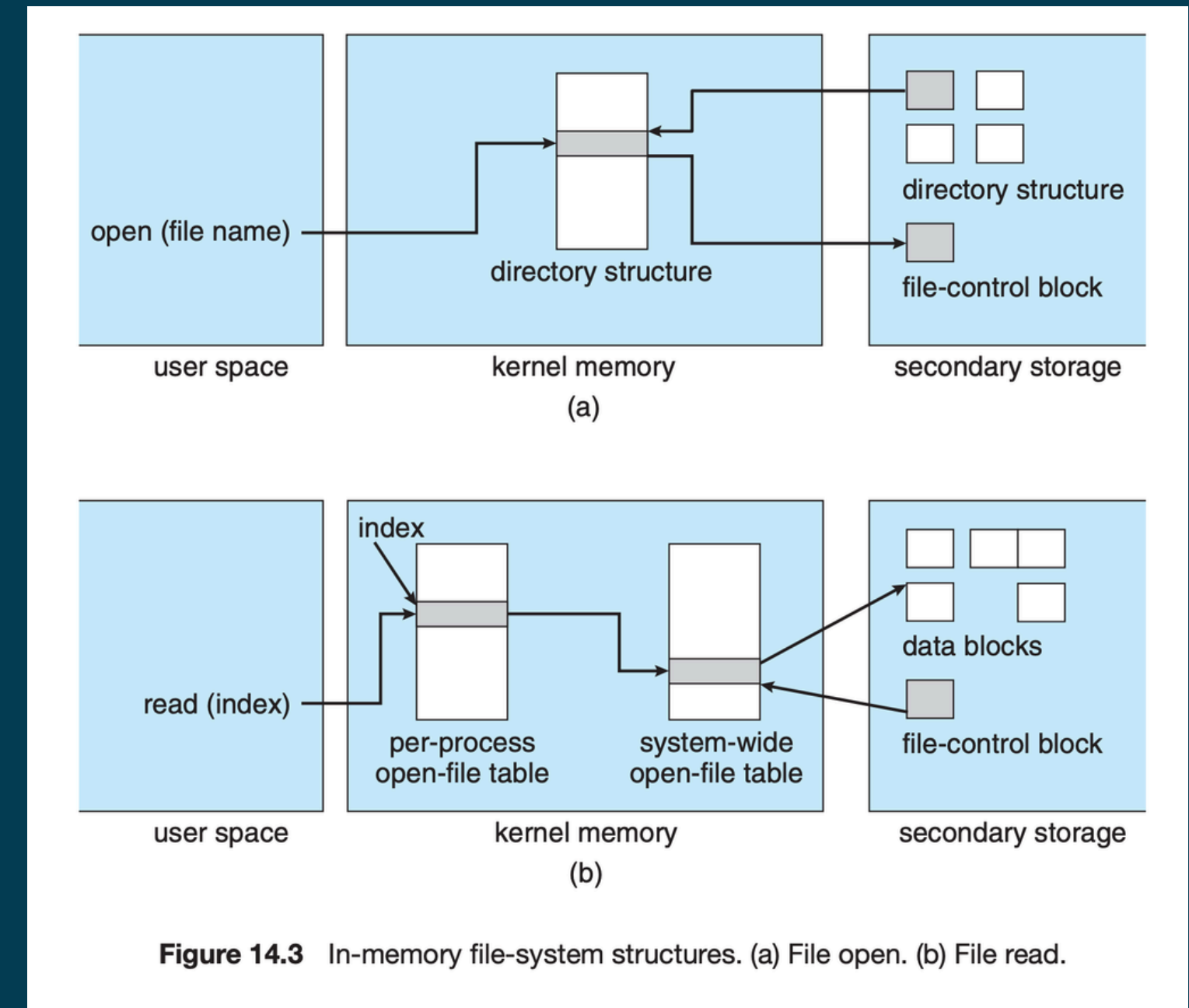
Leer un archivo desde un pendrive con FAT32 montado en un sistema Mac.

Gracias a VFS, se accede al archivo de forma transparente, sin importar el tipo de FS.



File Systems en Memoria Principal

- Un File System en memoria se monta directamente sobre la RAM, en lugar de un disco físico.
- Se utiliza cuando se requiere velocidad extrema de acceso, ya que la RAM es mucho más rápida que cualquier tipo de almacenamiento permanente.
- Ejemplos comunes:
 - /tmp en muchas distribuciones de Linux (temporal).
 - Sistemas embebidos o live-CDs que cargan todo el FS en memoria.
- Características:
 - Extremadamente rápido.
 - Los datos no persisten tras un reinicio (son volátiles).
 - Ideal para archivos temporales, cachés o testing.
- Tipos de FS en memoria:
 - tmpfs: muy usado en Linux; puede intercambiar a disco si falta RAM.
 - ramfs: siempre permanece en RAM, sin swap.



Fragmentacion

Al igual que en la memoria, los sistemas de archivos también enfrentan problemas de fragmentación, lo que puede afectar el rendimiento.

Fragmentación Interna:

- Ocurre cuando se asigna un bloque de tamaño fijo a un archivo, pero este no ocupa todo el espacio disponible en dicho bloque.

Resultado: Espacio desaprovechado dentro del bloque.

Fragmentación Externa:

- Sucede cuando hay suficiente espacio libre en el disco en total, pero está disperso en bloques no contiguos, impidiendo guardar archivos grandes de forma continua.

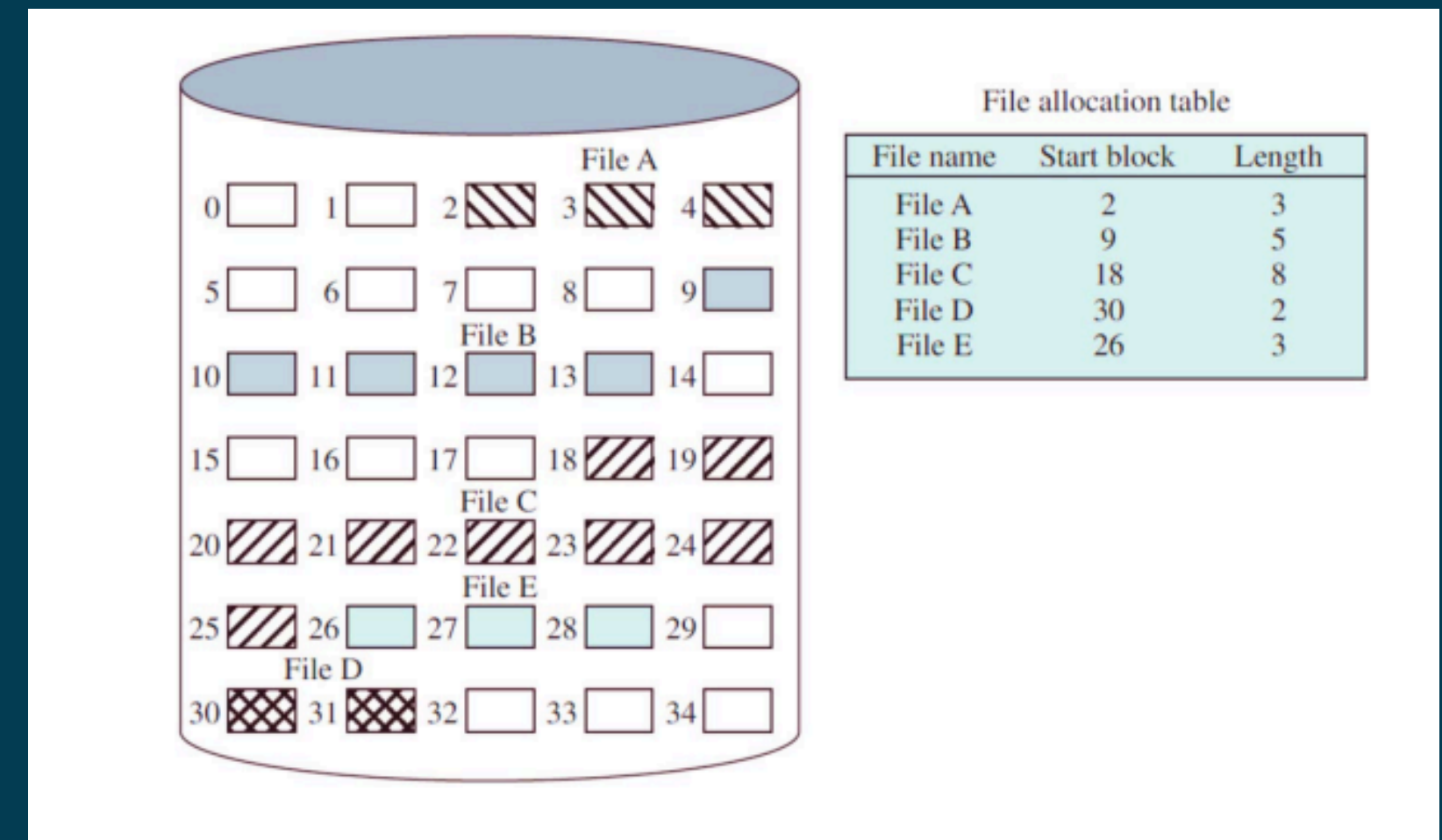
Resultado: Mayor tiempo de lectura/escritura y desorden.

Como se soluciona?

Mediante la asignación de bloques

Asignación Contigua

- Se asigna una secuencia continua de bloques a un archivo.
- Acceso rápido y eficiente a datos secuenciales, ya que los bloques están físicamente uno al lado del otro.
- Se mantiene una tabla con:
 - Bloque de inicio del archivo.
 - Tamaño total (en bloques).
- Problema: Fragmentación externa.
 - Con el tiempo, puede ser difícil encontrar espacio contiguo libre.
- Solución: Compactación del disco.
 - Reorganiza archivos para liberar espacio contiguo.
 - Menos costoso que compactar la RAM.



Asignación Enlazada

Cada archivo se almacena como una lista de bloques que pueden estar dispersos en el disco.

Estructura:

- Existe una tabla que guarda el bloque de inicio y el bloque final.
- Cada bloque contiene:
 - Los datos del archivo.
 - Un puntero al siguiente bloque en la lista.

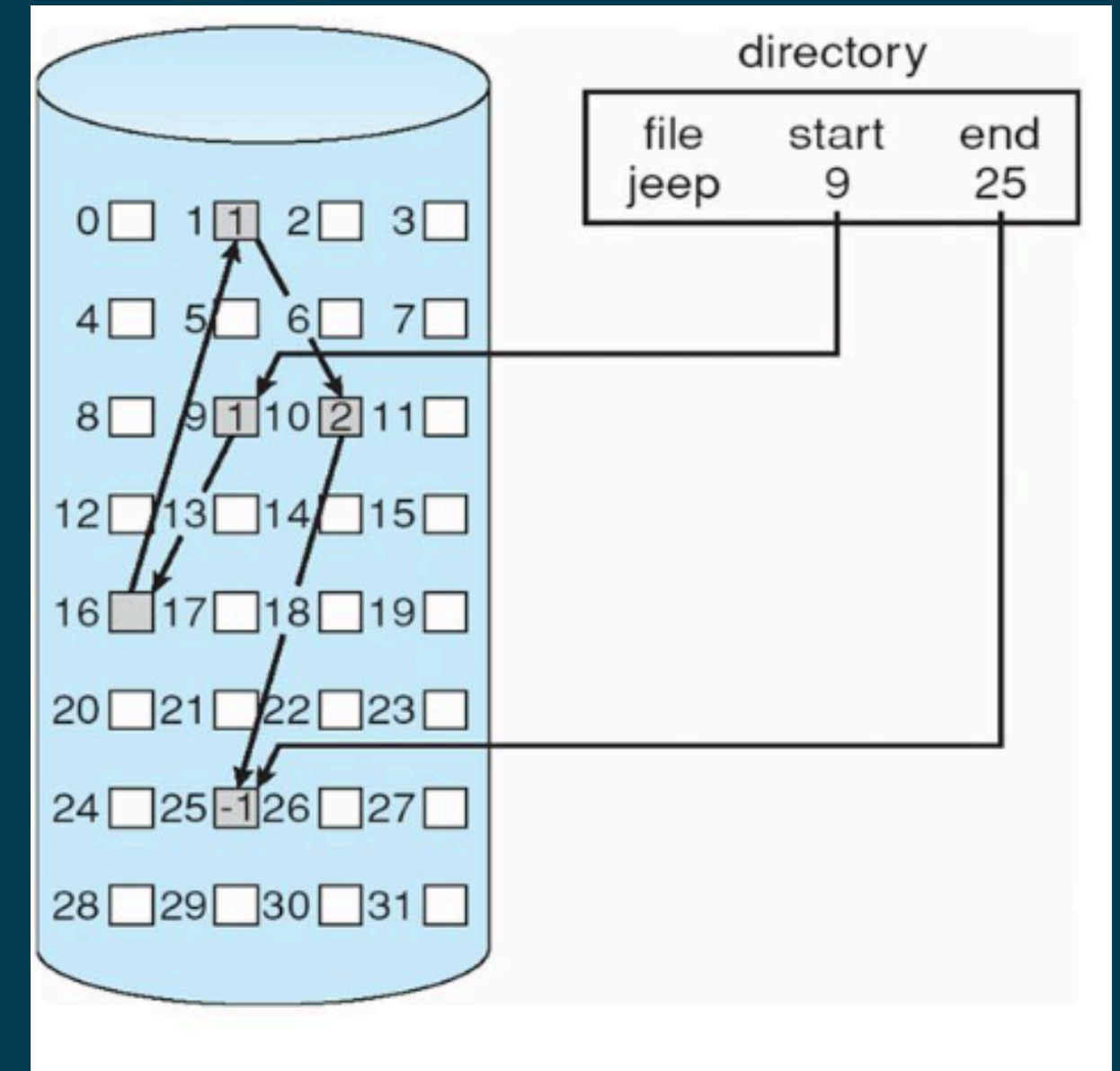
Ventajas:

- Los archivos pueden crecer de forma dinámica.
- No requiere bloques contiguos, lo que ayuda a evitar fragmentación externa.

Desventajas:

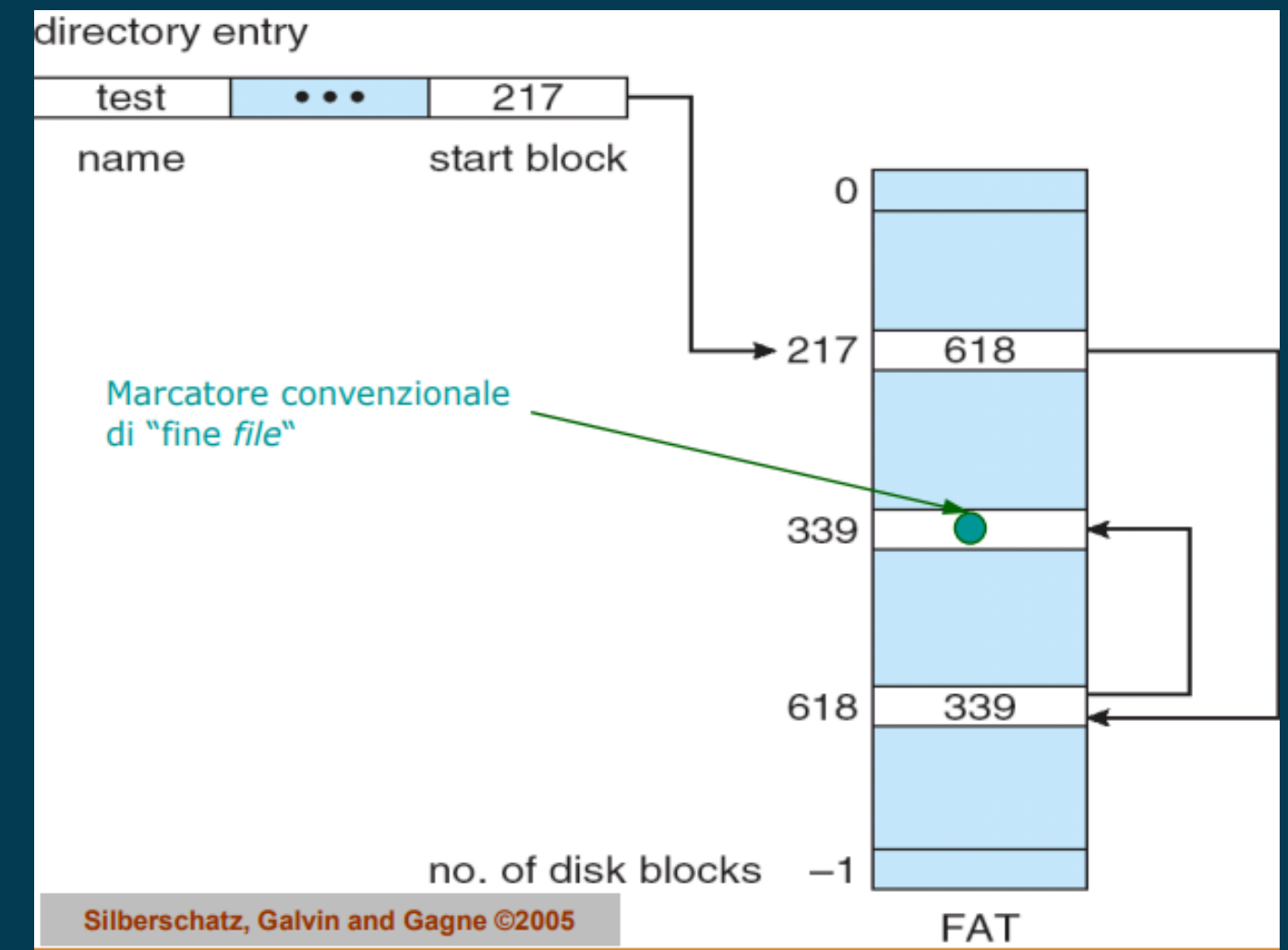
- No permite acceso aleatorio eficiente.
Para acceder a un bloque intermedio se debe recorrer la lista desde el inicio.
- Si un bloque se daña, se pierde el acceso a los bloques posteriores.

Es ideal para archivos que se acceden de forma secuencial.



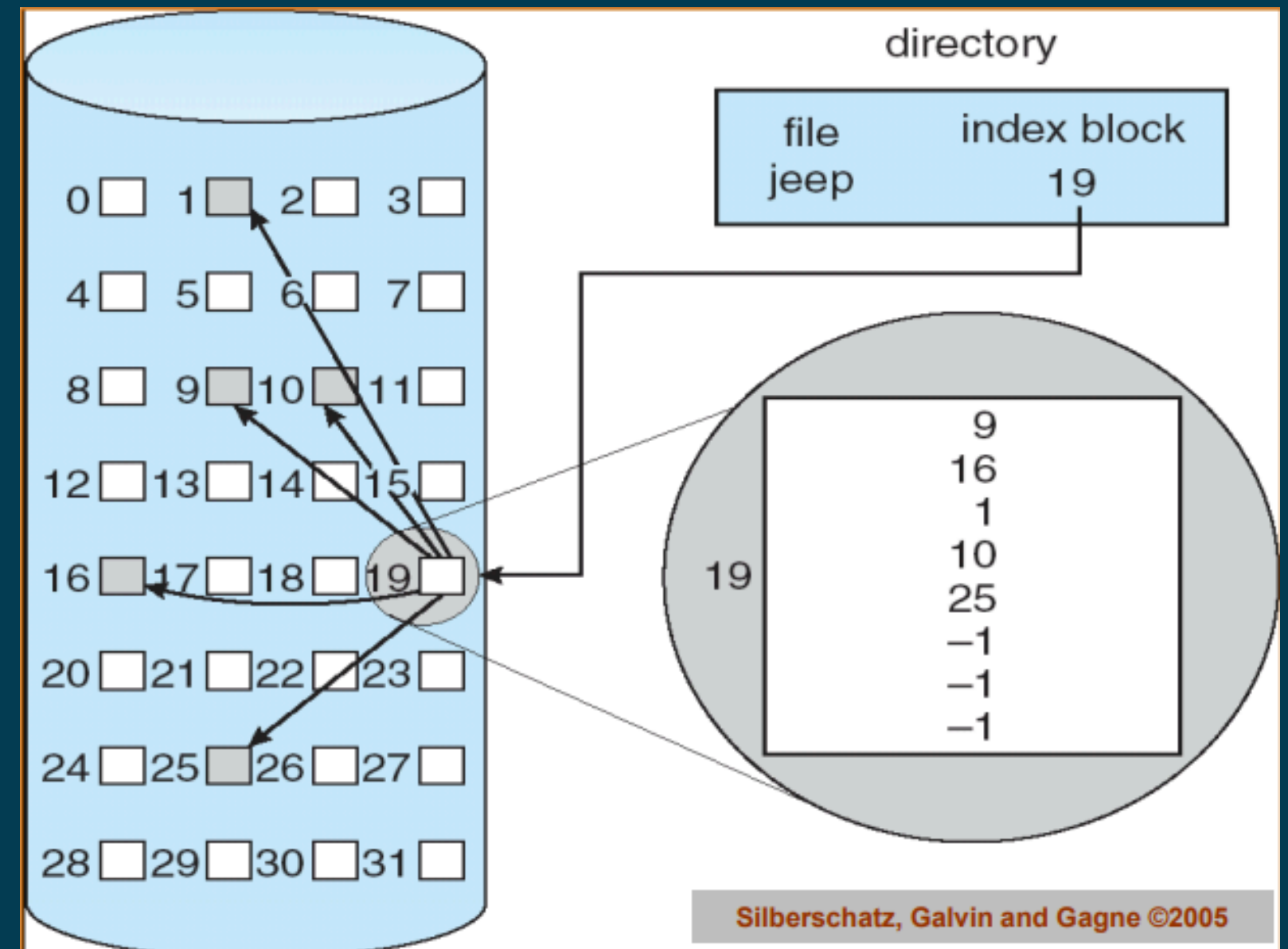
File Allocation Table (FAT)

- Es una variante mejorada de la asignación enlazada para gestionar archivos.
- Utiliza una tabla que registra el bloque inicial de cada archivo y los enlaces a los bloques siguientes, mejorando el acceso secuencial.
- El último bloque de un archivo contiene un valor especial (por ejemplo, -1) que indica el End Of File (EOF).
- Tiene limitaciones en el tamaño máximo de archivo según la versión:
- Por ejemplo, en FAT32, el tamaño máximo es de 4 GB (2^{32} bytes).



Asignación Indexada

- Cada archivo tiene un bloque índice que contiene punteros a todos los bloques de datos que lo componen.
- Permite acceso aleatorio eficiente a los datos, sin los problemas de fragmentación externa que afectan a otros métodos.
- El último bloque del archivo contiene un marcador de End Of File (EOF).
- Desventaja:
Requiere espacio adicional para almacenar el bloque índice de cada archivo, lo que puede aumentar el uso total de espacio en disco.



Tipos de Asignación Indexada

- Esquema enlazado

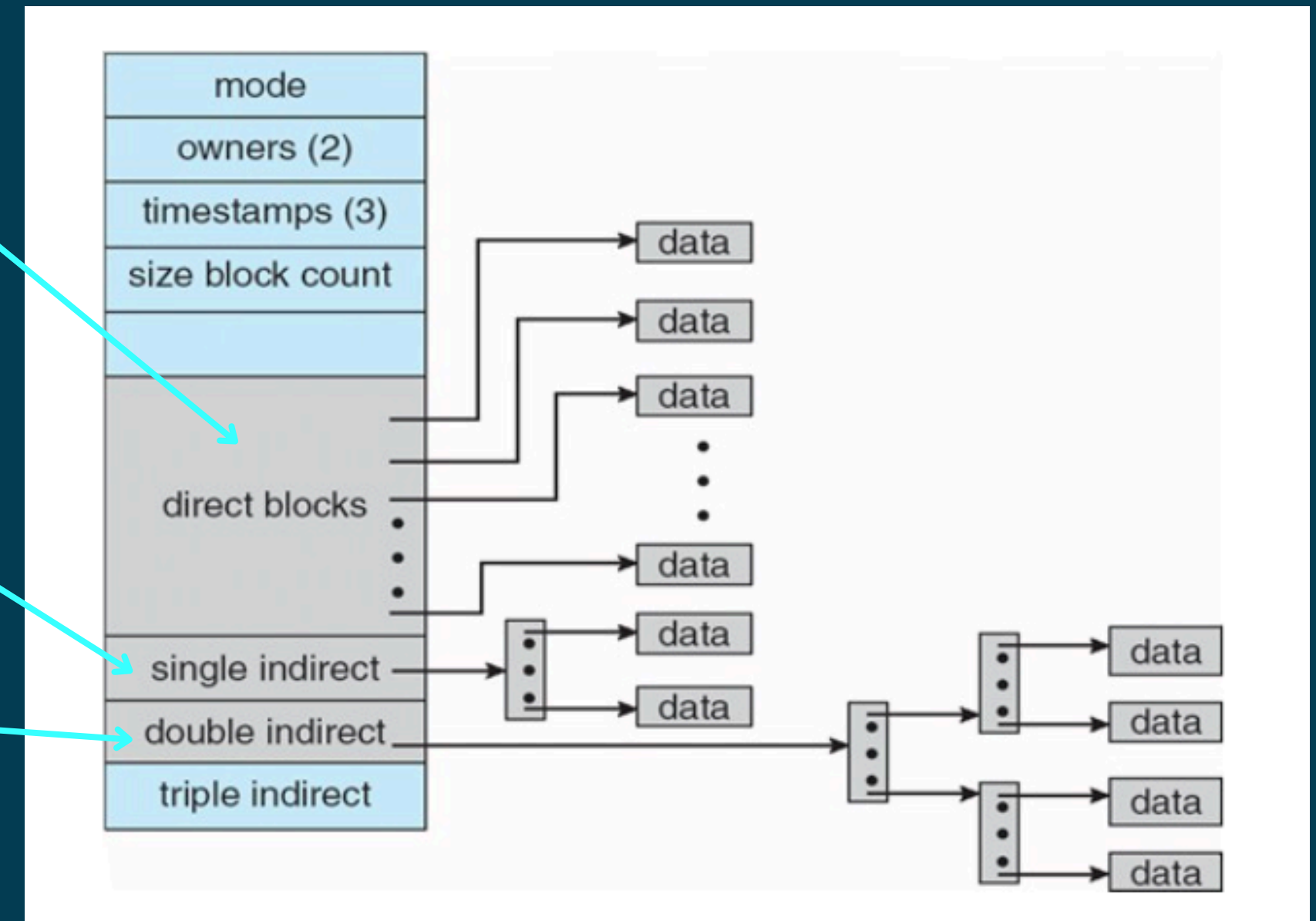
Utiliza una lista enlazada de bloques libres, donde cada bloque apunta al siguiente bloque libre disponible.

- Esquema multinivel

Emplea múltiples niveles de tablas de índices para administrar bloques libres de diferentes tamaños, optimizando la búsqueda y asignación.

- Esquema combinado

Integra listas enlazadas y tablas de índices para aprovechar las ventajas de ambos métodos, mejorando la flexibilidad y eficiencia en la gestión de bloques libres.



Como podemos encontrar espacio libre?

Bit Vector / Bitmap

- Es una serie de bits donde cada bit representa un bloque del disco.
- Un bit en 0 indica que el bloque está asignado.
- Un bit en 1 indica que el bloque está libre.
- Es fácil de implementar pero no escala bien en discos grandes debido a su tamaño fijo y recorrido secuencial.

Lista Enlazada

- Utiliza punteros para enlazar los bloques libres entre sí.
- Puede ser menos eficiente por la necesidad de mantener una estructura adicional.
- Sin embargo, es más escalable y flexible para manejar espacios libres dispersos.

Ejercicio: File System

Supongamos que tenemos un disco con 20 bloques, enumerados del 0 al 19, y que necesitamos almacenar tres archivos en este disco:

Archivo A: 5 bloques | Archivo B: 3 bloques | Archivo C: 6 bloques

Se considerarán bloques completos para almacenar los bloques de los archivos y para marcar los índices (si es que es necesario) de dichos archivos. Determine la asignación de bloques para administrar el almacenamiento (de estos archivos) en el disco considerando:

- Asignación Contigua.
- Asignación Enlazada bajo esquema de punteros a bloques intercalados.
- Asignación Indexada.

*No olvide que si necesita marcar un índice, esto considera un bloque en el disco